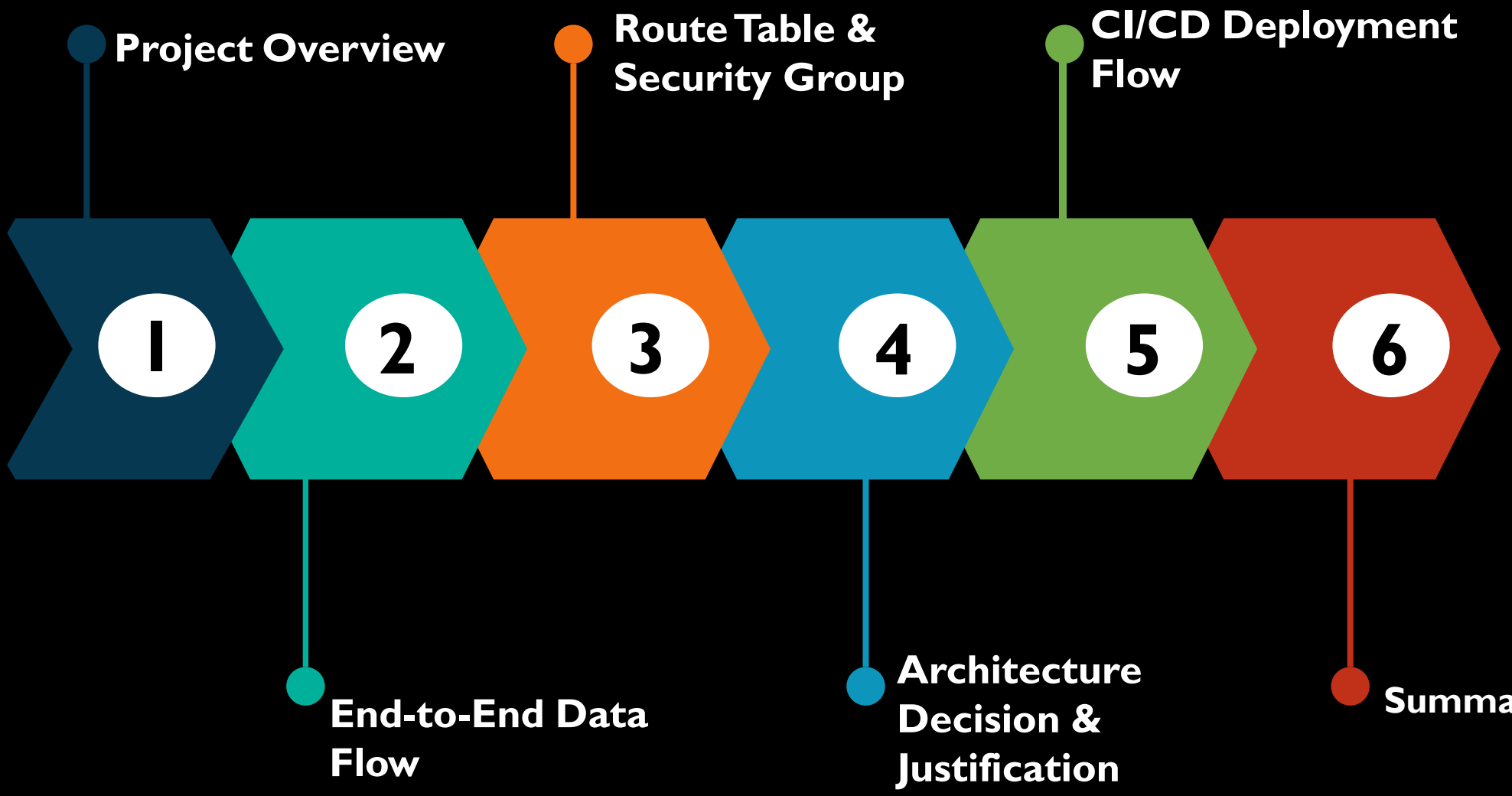
Luxe-Ecommerce | ADR

Architecture Decision Review

- By Chukwudi Brian Chima

Table of Contents



A Project Overview

Luxe | Business Context | Problem Statement

Project Overview

Business Context:

Luxe is a modern e-commerce platform designed to deliver a fast, secure, and scalable online shopping experience. The application operates in a highly competitive environment where:

- Page load speed directly impacts conversion rates
- Downtime leads to immediate revenue loss
- Traffic patterns are unpredictable due to promotions, sales, and seasonal demand
- Security and data protection are critical to user trust

The primary goal of this project is to demonstrate a production-grade, well-architected e-commerce system that aligns with AWS best practices.

Problem Statement

Scalability & Elasticity

Efficiently handle a rapidly increasing or decreasing customer traffic.

Security

Integrate securely with APIs. Ensure data security either at rest or in transit. Protect network from cyberattacks.

Cost Efficiency

Ensure best performance on budget, without any performance or security trade off.

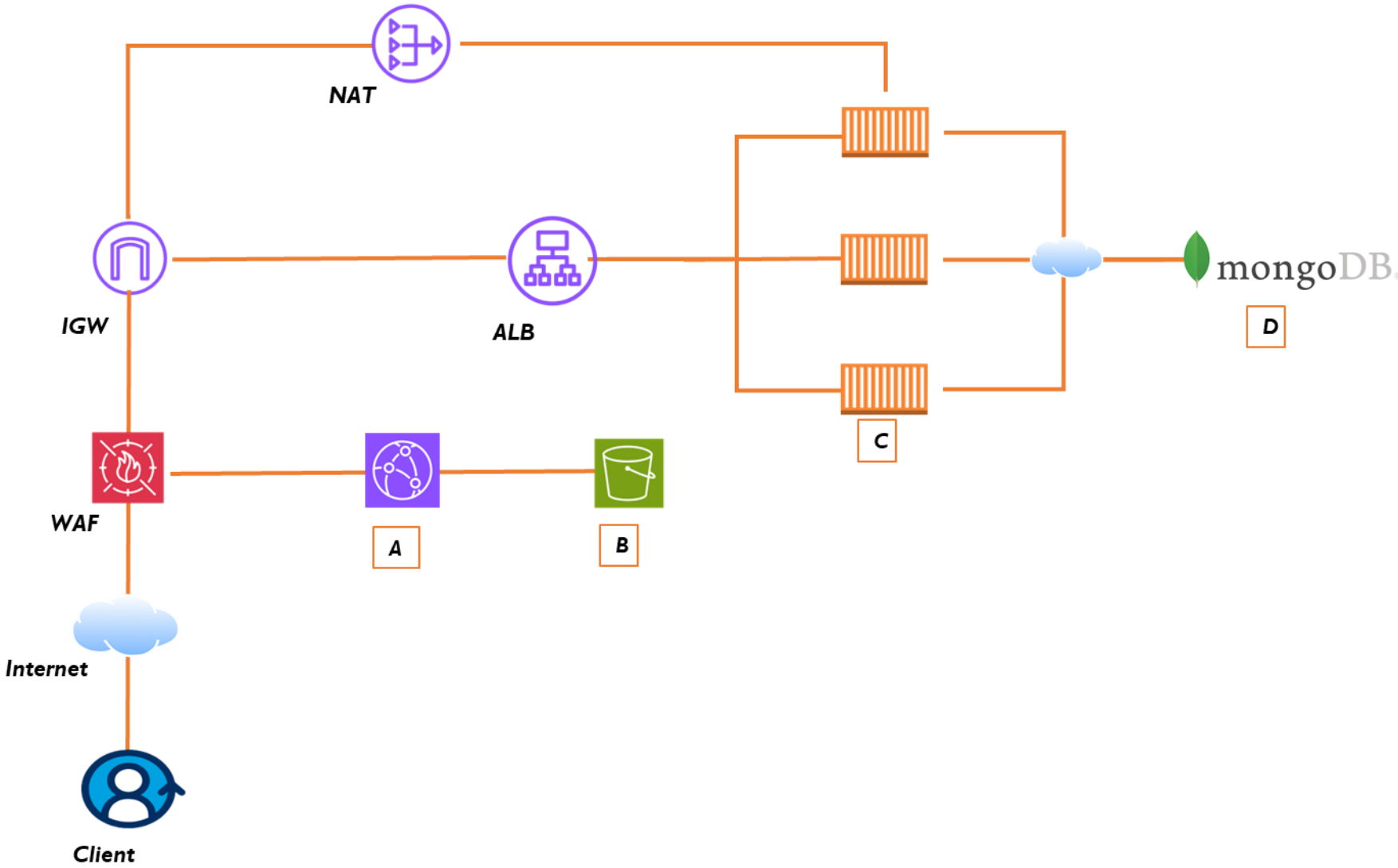
Operational Excellence

Automate infrastructure provisioning, scaling, centralized monitoring. consistent, low operational overhead.

B Proposed Solution

Luxe | Data Flow Diagram| Network Diagram

Proposed Data Flow Diagram



Architecture Decision:

- Networking – CloudFront, ALB, IGW, NAT
- Security – WAF, NACLs, Security Groups
- Storage – S3
- Compute – ECS Fargate
- Container Registry - ECR

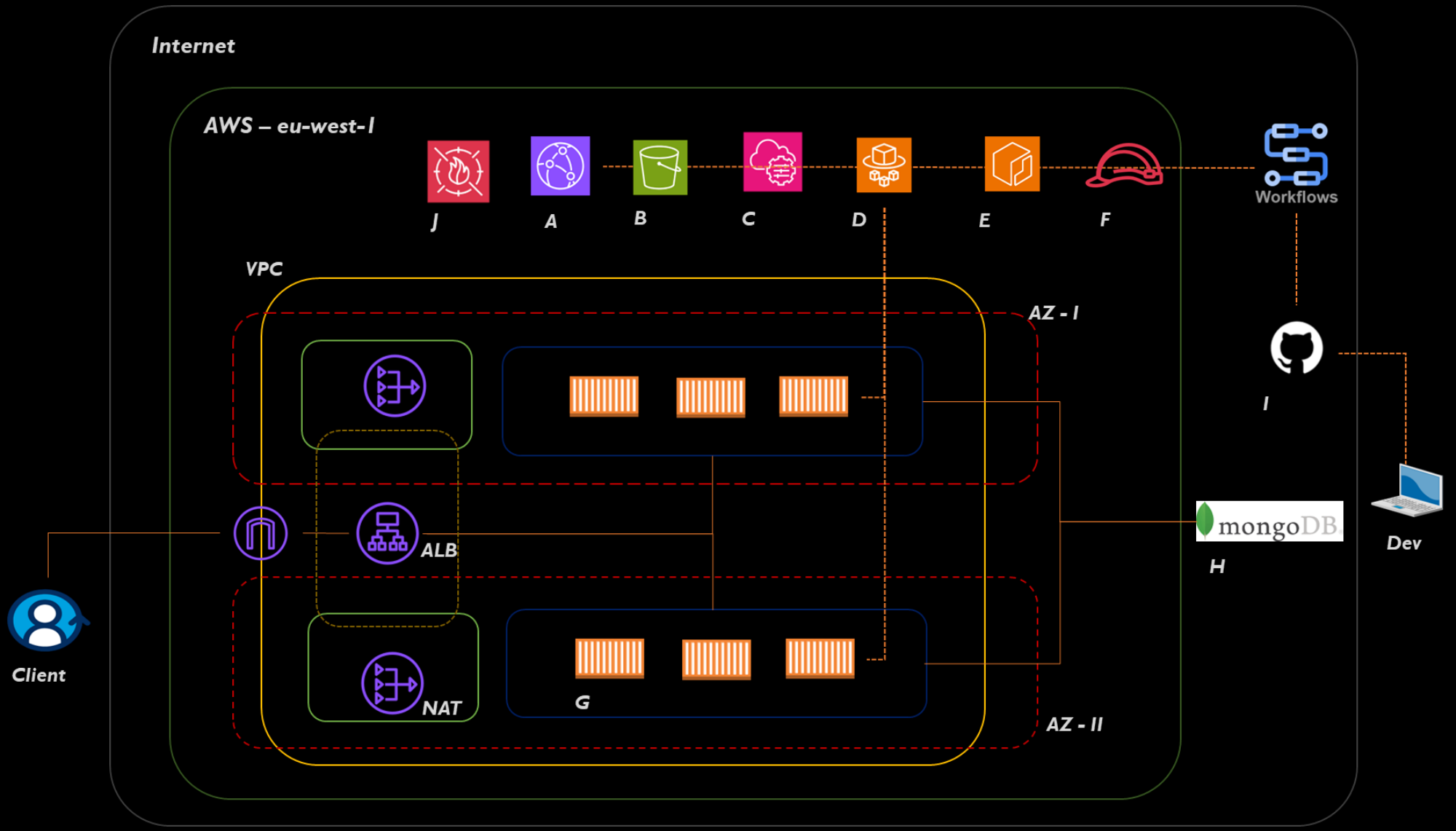
Key Components:

A. CloudFront
B. S3 Bucket
C. ECS Containers
D. 3rd Party Mongo DB Atlas.

Key Concerns:

- **Availability**
- **Elasticity**
- **Scalability**
- **Security**

Proposed Network Diagram



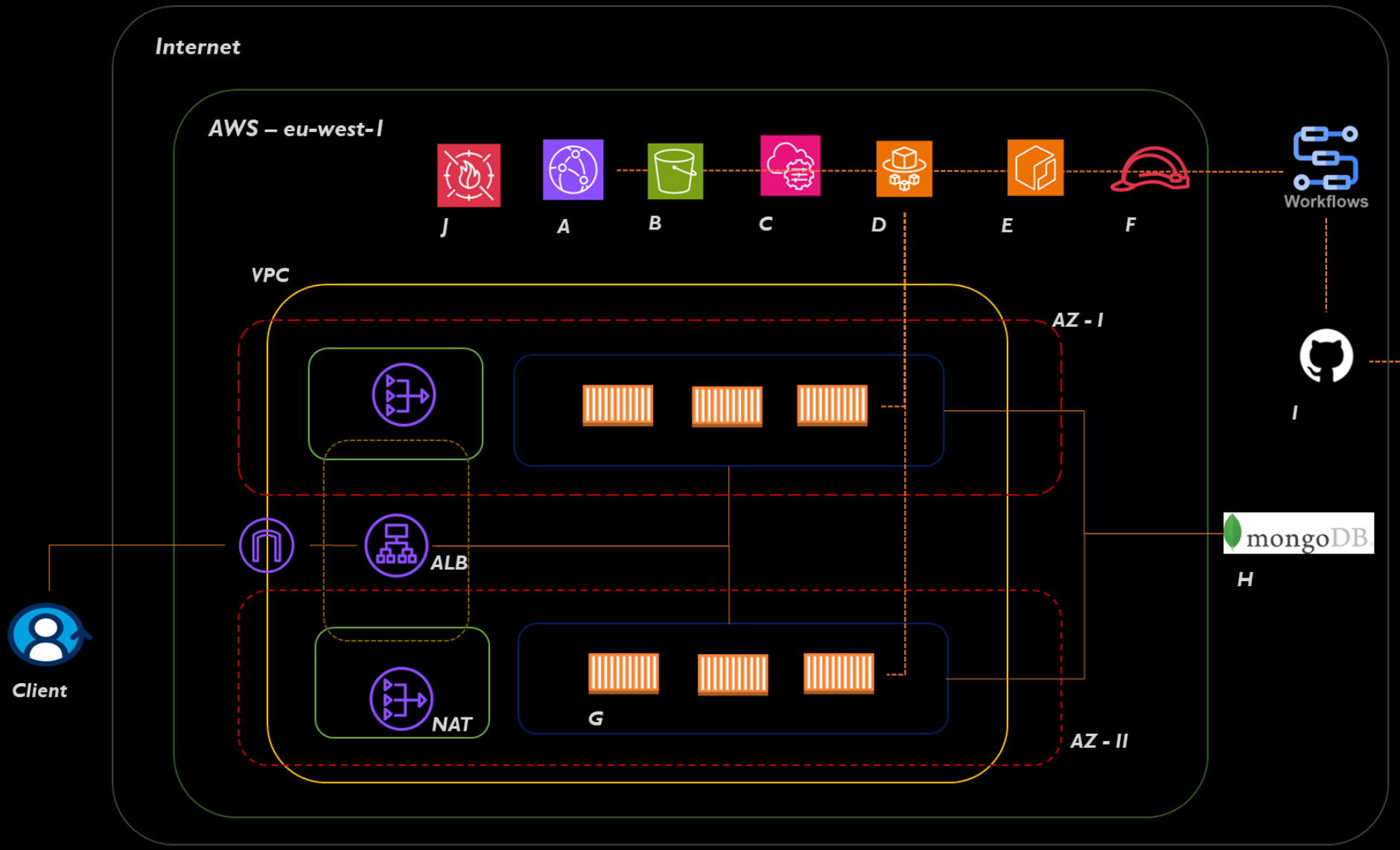
LEGEND

A. CloudFront
B. S3
C. SSM Parameter
D. ECS Fargate
E. ECR
F. IAM Role
G. ECS Container(Service)
H. 3rd Party MongoDB Atlas
I. GitHub
J. WAF

AZ– Availability Zone

Public Subnet
Private Subnet

Proposed Network Diagram



Availability

1. Auto Scaling- ECS Fargate
2. Multi Az deployment

Security

1. AWS WAF, security group/NACL at respective levels
2. IAM policies(POLP)
3. In-built DDOS protection(CloudFront)

Performance

1. Load Management- ALB
2. CloudFront

Route Table & Security Group

Luxe |

Route Table

	ROUTER-NAME	ASSOCIATION	DESTINATION	TARGET	COMMENT
1	routerA	All subnet in the VPC	VPC CIDR block	Local	This handles all the traffic with a destination IP in VPC CIDR block
2	routerA	Public subnet	0.0.0.0/0	internet gateway	This handles all internet bound traffic from public subnet
3	routerA	Private subnet	0.0.0.0/0	NAT gateway	This handles all internet bound traffic from the private subnet.

Security Group

Resource	Protocol	Port	Traffic	Rule	Source
Load Balancer(LB)	HTTP	80	Inbound	Allow	0.0.0.0/0
ECS Containers	HTTP	3000	Inbound	Allow	LB SG
LB, ECS Containers	ANY	ANY	Outbound	Allow	0.0.0.0/0

D Architecture Decision & Justification

Luxe | Trade-offs & Considerations | Future Improvements

Architecture Decisions and Justifications

- Decision: Host the frontend as static assets in Amazon S3 and distribute them via Amazon CloudFront
- Justification: S3 provides durable, highly available object storage for frontend assets. CloudFront delivers content from edge locations close to users, reducing latency and improving security with built-in DDoS protection. This ensures that frontend availability is largely independent of backend health.

- Decision: Implement backend functionality as containerized microservices running on Amazon ECS Fargate.
- Justification: Fargate removes the need to manage EC2 instances. Each service can scale independently based on demand. Stateless services allow for easy replacement and recovery. This approach enables Luxe to evolve services independently while maintaining operational simplicity.

Architecture Decisions and Justifications

- Decision: Use an Application Load Balancer as the single point of entry for backend API traffic.
- Justification: Path-based routing enables clean separation of services (e.g., /api/products, /api/users). Health checks ensure traffic is only sent to healthy ECS tasks. Supports horizontal scaling and zero-downtime deployments.

- Decision: Apply layered security(Defense in depth) using IAM roles, private networking, and AWS WAF.
- Justification: ECS services run in private subnets with no public IPs. IAM roles enforce principle of least-privilege per service. AWS WAF protects both CloudFront and ALB from common web attacks. Rate limiting per IP reduces abuse and brute-force attempts. Security is embedded into the architecture rather than added as an afterthought.

- Decision: AWS ECR for Image Storage
- Justification: Native AWS integration for permissions, networking, and authentication. Minor cost per repository/storage.

Architecture Decisions and Justifications

- Decision: GitHub action CI/CD Instead of AWS CodePipeline
- Justification: Flexible, script-driven pipelines. Full control for deployment logic.

- Decision: Terraform for IaC
- Justification: Reproducible environments. Modular, auditable, scalable to multi-env setups.

Trade-Offs & Considerations

- Reduced Control: Using ECS Fargate simplifies operations by removing server management but offers less low-level control compared to self-managed EC2-based ECS or Kubernetes.

- CloudFront & ALB Complexity: Separating frontend and backend traffic improves scalability(backend services scale independently) and security but increases architectural complexity and requires careful routing and cache behavior configuration.

- Terraform State Management: Remote state in S3 with locking provides safety and collaboration benefits but requires strict IAM controls to prevent accidental destructive actions.

Trade-Offs & Considerations

- Microservices Overhead: While microservices improve scalability and team autonomy, they introduce operational overhead such as inter-service communication, monitoring, and versioning challenges.

- WAF and Security Controls: Adding WAF, rate limiting, and strict IAM roles improves security posture, but may introduce latency and requires ongoing tuning to avoid blocking legitimate traffic.

- OIDC-Based CI/CD Access: Short-lived credentials via GitHub OIDC improve security significantly but require precise trust policies and careful separation of roles per workload.

Future Improvements

- Migrate 3rd party managed Database to AWS: Migrate MongoDB atlas database to a compatible AWS service(DocumentDB) using AWS Data Migration Service. This ensure private connection to DB in VPC as well as reduce latency, instead of unsecure high latency connection over the internet.

- Blue/Green or Canary Deployments: Introduce ECS blue/green deployments using AWS CodeDeploy or weighted target groups to reduce deployment risk and enable safer rollouts with automatic rollback.

- Cost Optimization Enhancements: Introduce Fargate Spot for non-critical services(running stateful workload) , lifecycle rules for S3 objects, and more aggressive CloudFront caching policies.

- Advanced Traffic Management: As the number of microservices grows, tools like AWS App Mesh or API Gateway could provide better observability, retries, and traffic shaping between services.

Future Improvements

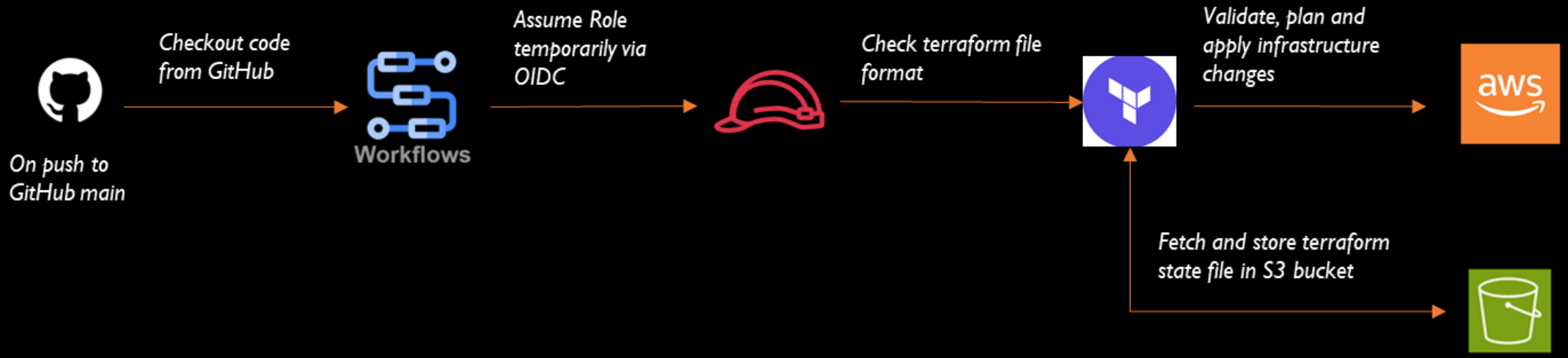
- Caching at the API Layer: Add response caching using CloudFront, API Gateway, or an in-memory cache like Redis (ElastiCache) to reduce backend load and improve latency for read-heavy endpoints.

- Multi-Region Resilience: Extend the architecture to multiple regions using Route 53 health checks and failover routing to improve availability and disaster recovery posture.

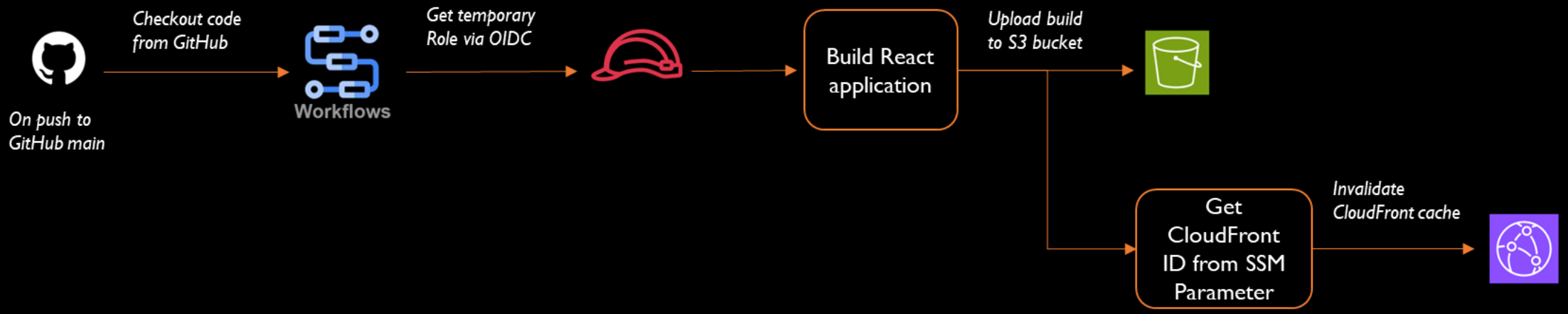
CI/CD Deployment Flow

Luxe | Infrastructure | Frontend | Backend

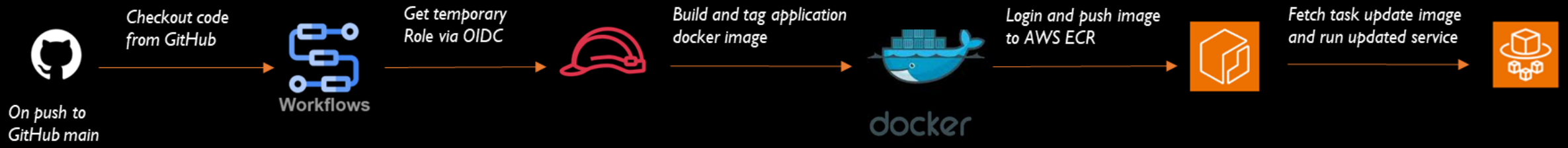
Infrastructure CI/CD Pipeline Flow Diagram



Frontend CI/CD Pipeline Flow Diagram



Backend CI/CD Pipeline Flow Diagram



Summary

Luxe |

Summary

- Luxe is a cloud-native e-commerce platform designed for scalability, security, and operational simplicity. The frontend is delivered as a static React application via Amazon S3 and CloudFront for global performance and built-in DDoS protection, while the backend is implemented as independent microservices running on Amazon ECS Fargate behind an Application Load Balancer. Infrastructure is fully defined using Terraform, and deployments are automated with GitHub Actions using OIDC for secure, keyless access to AWS. The architecture follows AWS Well-Architected principles, enabling fast iteration, high availability, and clear separation between infrastructure and application delivery while remaining cost-efficient and production-ready.